Rapport de Travaux Pratiques

Architectures Logicielles

Module :	Architectures Logicielles
Encadrant :	Pr. BE ELBAGHAZAOUI
Établissement :	ENSA Beni Mellal
Année :	2025 – 2026
Étudiant(e) :	ILYAS MACHHOUL
N° Apogée :	23012747

Chapitre 1

TP1 — Application Monolithique avec Spring Boot

1.1 Objectifs

Objectifs du TP

- Créer une application Spring Boot monolithique simple.
- Développer une API REST CRUD complète pour la gestion de produits.
- Comprendre l'architecture en couches : Entity, Repository, Service, Controller.
- Découvrir `@RepositoryRestResource` pour simplifier le code.
- Identifier les avantages et les limites d'une architecture monolithique.

1.2 Outils et Prérequis

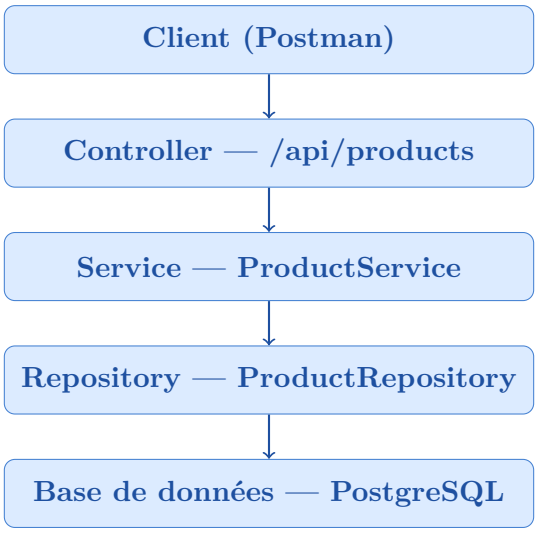
Outil	Version / Détail
JDK	17+
Maven	3.8+
IDE	IntelliJ IDEA / Eclipse / VS Code
Base de données	PostgreSQL ou MySQL
Client REST	Postman
Générateur	start.spring.io

Table 1.1 – Outils nécessaires pour le TP1

1.3 Architecture du Système

Architecture

L'application suit une architecture **monolithique en couches** classique :



Flux de données : Postman → Controller → Service → Repository → PostgreSQL

1.3.1 Structure du projet

```
com.ecommerce.monolith
          product
              controller
                    ProductController.java
              service
                    ProductService.java
              repository
                    ProductRepository.java
              model
                  Product.java
```

Listing 1.1 – Structure du projet Spring Boot

1.4 Code Source Important

1.4.1 Entité Product

```java
@Entity
@Table(name = "products")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
public class Product {
```

```
6
7       @Id
8       @GeneratedValue(strategy = GenerationType.IDENTITY)
9       private Long id;
10
11      @NotBlank(message = "Product name is required")
12      @Column(nullable = false)
13      private String name;
14
15      @Column(length = 1000)
16      private String description;
17
18      @NotNull(message = "Price is required")
19      @Positive(message = "Price must be greater than 0")
20      @Column(nullable = false, precision = 10, scale = 2)
21      private BigDecimal price;
22
23      @NotNull(message = "Stock is required")
24      @PositiveOrZero(message = "Stock cannot be negative")
25      @Column(nullable = false)
26      private Integer stock;
27  }
```

Listing 1.2 – Product.java — Entité JPA

1.4.2 Repository

```
1  @Repository
2  public interface ProductRepository extends
       JpaRepository<Product, Long> {
3      List<Product> findByNameContainingIgnoreCase(String
           name);
4  }
```

Listing 1.3 – ProductRepository.java — Version classique

1.4.3 Service

```
1  @Service
2  @RequiredArgsConstructor
3  @Transactional
4  public class ProductService {
5
6      private final ProductRepository repository;
7
8      @Transactional(readOnly = true)
9      public List<Product> getAll() {
10         return repository.findAll();
11     }
12
```

```java
    @Transactional(readOnly = true)
    public Product getById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException(
                "Product not found with id: " + id));
    }

    public Product create(Product product) {
        return repository.save(product);
    }

    public Product update(Long id, Product details) {
        Product product = getById(id);
        product.setName(details.getName());
        product.setDescription(details.getDescription());
        product.setPrice(details.getPrice());
        product.setStock(details.getStock());
        return product; // save inutile si entite managed
    }

    public void delete(Long id) {
        Product product = getById(id);
        repository.delete(product);
    }
}
```

Listing 1.4 – ProductService.java — Logique métier

1.4.4 Controller REST

```java
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
@Validated
public class ProductController {

    private final ProductService service;

    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        return ResponseEntity.ok(service.getAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getById(@PathVariable
       Long id) {
        return ResponseEntity.ok(service.getById(id));
    }

    @PostMapping
    public ResponseEntity<Product> create(
```

```
            @Valid @RequestBody Product product) {
        Product created = service.create(product);
        return
           ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> update(
            @PathVariable Long id,
            @Valid @RequestBody Product product) {
        return ResponseEntity.ok(service.update(id,
           product));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long
       id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Listing 1.5 – ProductController.java — API REST

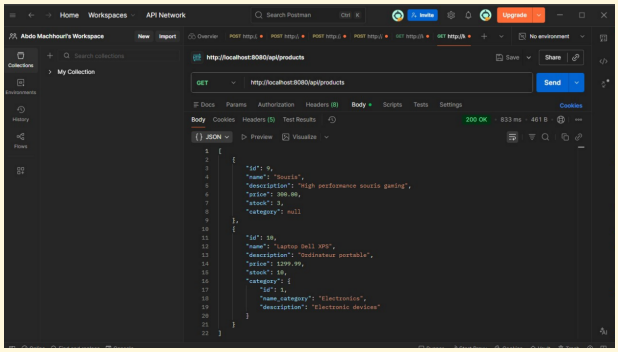
1.5 Tests avec Postman

Méthode	URL	Description
GET	/api/products	Liste tous les produits
GET	/api/products/{id}	Récupère un produit par ID
POST	/api/products	Crée un nouveau produit
PUT	/api/products/{id}	Modifie un produit existant
DELETE	/api/products/{id}	Supprime un produit

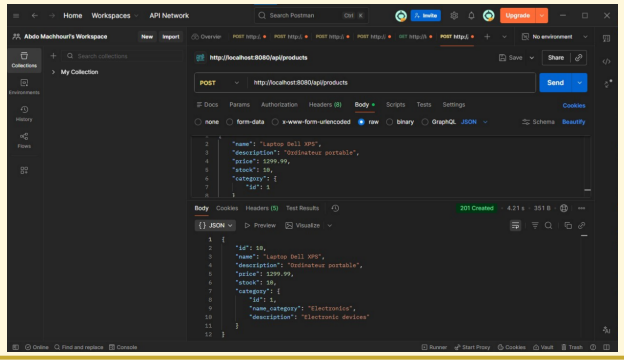
Table 1.2 – Endpoints de l'API REST — TP1

1.5.1 Captures d'écran Postman

Capture Postman — GET /api/products — Liste des produits



Capture Postman — POST /api/products — Création d'un produit



1.6 Résumé du TP1

Résumé

Le TP1 introduit les bases de Spring Boot à travers une application monolithique simple. Les points clés à retenir sont :

- **Architecture en couches** : séparation claire entre Controller, Service, Repository et Model.
- **Spring Data JPA** : abstraction de la persistance grâce à `JpaRepository` qui génère automatiquement les requêtes CRUD.
- **Lombok** : réduit le code boilerplate (`@Getter`, `@Setter`, `@Builder`, etc.).
- **Validation** : contraintes automatiques via `@NotBlank`, `@Positive`, `@Valid`.
- **Limites du monolithe** : couplage fort entre composants, difficulté à scaler indépendamment, déploiement monobloc.